# AEGIS: Adaptive Encryption and Guarded Integrated Secret-Key System

Formal Mathematical Specification, Finite-Field Threshold Algebra, and Key-Committed AEAD Architecture

Project AEGIS Cryptographic Engineering Specification

October 2026

**Abstract**

Conventional symmetric key storage forces an architectural trade-off between single-point-of-failure availability risks and key-replication exposure risks. Furthermore, naive applications of Shamir's $(k, n)$ Secret Sharing (SSS) suffer from unauthenticated reconstruction: any $k$ shares algebraically interpolate a polynomial, silently producing a corrupted secret if any share is damaged or forged. This paper presents the formal mathematical and systems specification of **AEGIS** (*Adaptive Encryption and Guarded Integrated Secret-Key System*), implemented in memory-safe Rust (`#![forbid(unsafe_code)]`). AEGIS integrates prime finite-field threshold algebra over the eighth Mersenne prime $\mathbb{F}_p = \mathrm{GF}(2^{31}-1)$, domain-separated key derivation via HKDF-SHA256 (RFC 5869), and Authenticated Encryption with Associated Data (AEAD) via ChaCha20-Poly1305 (RFC 8439). We prove the absence of 64-bit integer overflow in $\mathbb{F}_p$, prove unique polynomial reconstruction via Vandermonde matrix invertibility, establish a two-layer verifiable reconstruction trapdoor with false-acceptance probability bounded by $2^{-240}$ (algebraic) and $2^{-256}$ (cryptographic commitment), and analyze the system's side-channel and operational boundaries.

## Contents

**1 System Overview** **2**

**2 Threat Model** **2**

**3 Finite-Field Arithmetic over $\mathbb{F}_p$ ($p = 2^{31}-1$)** **3**
4.1 Overflow-Free 64-Bit Arithmetic Guarantee . . . . . 3
4.2 Fermat's Little Theorem and Multiplicative Inverses . . . . . 3
4.3 Unbiased Rejection Sampling . . . . . 4

**5 Polynomial Construction and Horner's Evaluation** **4**

**6 Lagrange Interpolation at $x = 0$** **4**

**7 Vandermonde Matrix Representation** **5**

**8 Key Derivation Hierarchy (HKDF-SHA256)** **5**

**9 Authenticated Encryption (ChaCha20-Poly1305)** **6**
9.1 ChaCha20 Stream Cipher . . . . . 6
9.2 Canonical AAD Binding and Poly1305 Evaluation . . . . . 6

**10 Secret-Sharing Security Proof & Algebraic Trapdoor** **6**

**11 Implementation Details in Rust** **7**

**12 Security Analysis** **7**

**13 Test Methodology** **7**

**14 Limitations** **8**

**15 Conclusion** **8**

# 1 System Overview

Let $\mathcal{K} = \{0,1\}^{256}$ denote the space of 256-bit (32-byte) master keys. AEGIS is structured around four composable mathematical mappings:

(1) **Entropy Generation:** $K_{\text{master}} \xleftarrow{\$} \mathcal{K}$ sampled uniformly from the operating system Cryptographically Secure Pseudo-Random Number Generator (CSPRNG), rejecting the degenerate zero vector $0^{256}$.

(2) **Domain-Separated Key Hierarchy (HKDF-SHA256):** A deterministic, one-way derivation mapping $K_{\text{master}}$ to an orthogonal pair $(K_{\text{enc}}, C_{\text{key}}) \in \{0,1\}^{256} \times \{0,1\}^{256}$, where $K_{\text{enc}}$ is the symmetric AEAD subkey and $C_{\text{key}}$ is a public cryptographic commitment to $K_{\text{master}}$.

(3) **Threshold Decomposition over $\mathbb{F}_p$:** A randomized $(k, n)$ splitting algorithm ($2 \le k \le n \le 255$) that partitions $K_{\text{master}}$ into $m = 16$ sixteen-bit chunks $s_0, \ldots, s_{15} \in [0, 2^{16} - 1] \subset \mathbb{F}_p$ and evaluates 16 random degree-$(k-1)$ polynomials over $\mathbb{F}_p$ ($p = 2^{31} - 1$) at non-zero abscissas $x_i \in \{1, \ldots, n\}$.

(4) **Key-Committed AEAD Envelope (ChaCha20-Poly1305):** An authenticated encryption engine that binds the protocol header, $C_{\text{key}}$, and user-supplied Associated Data (AAD) into a canonical transcript authenticated by a 128-bit Poly1305 MAC over $\mathbb{F}_{2^{130}-5}$.

| Symbol | Domain / Value | Description |
|---|---|---|
| $p$ | $2^{31} - 1 = 2,147,483,647$ | Eighth Mersenne prime modulus ($M_{31}$) |
| $\mathbb{F}_p$ | $\{0, 1, \ldots, p-1\}$ | Prime finite field (Galois Field GF($p$)) |
| $\mathbb{F}_p^*$ | $\{1, 2, \ldots, p-1\}$ | Multiplicative group of non-zero elements in $\mathbb{F}_p$ |
| $K_{\text{master}}$ | $\{0,1\}^{256} \setminus \{0^{256}\}$ | 256-bit high-entropy root secret key |
| $K_{\text{enc}}$ | $\{0,1\}^{256}$ | HKDF-derived ChaCha20-Poly1305 subkey |
| $C_{\text{key}}$ | $\{0,1\}^{256}$ | HKDF-derived public master-key commitment |
| $k, n$ | $\mathbb{Z} \cap [2, 255],\ k \le n$ | Recovery threshold $k$ and total shares $n$ |
| $m$ | 16 | Number of 16-bit (`u16`) chunks per 256-bit key |
| $N$ | $\{0,1\}^{96}$ | 96-bit (12-byte) CSPRNG AEAD nonce |
| $T$ | $\{0,1\}^{128}$ | 128-bit (16-byte) Poly1305 authentication tag |

Table 1: Summary of formal notation used throughout the AEGIS specification.

# 2 Threat Model

We consider a probabilistic polynomial-time (PPT) active adversary $\mathcal{A}_{\text{comp}}$ with full read/write control over stored ciphertexts, envelope headers, andAssociated Data (the IND-CCA2 / INT-CTXT setting), as well as an computationally unbounded adversary $\mathcal{A}_{\text{info}}$ who compromises at most $k-1$ threshold shares.

- **Threshold Compromise Bound:** $\mathcal{A}_{\text{info}}$ may adaptively select any subset $I \subset \{1, \ldots, n\}$ of shares with $|I| \le k-1$. The scheme must guarantee zero mutual information: $I(K_{\text{master}}; \{S_i\}_{i \in I}) = 0$.
- **Active Share Tampering:** $\mathcal{A}_{\text{comp}}$ may modify share evaluations $y_{i,r} \in \mathbb{F}_p$, inject duplicate share coordinates $x_i = x_j$, or substitute shares generated from an independent key $K'_{\text{master}} \neq K_{\text{master}}$.

- **Envelope Header Forgery:** $\mathcal{A}_{\text{comp}}$ may flip arbitrary bits in $(C_{\text{key}}, N, C, T, \text{AAD})$ attempting to induce decryption of tampered plaintext or bypass key commitment verification.

# 3 Finite-Field Arithmetic over $\mathbb{F}_p$ $(p = 2^{31} - 1)$

**Definition 4** (Prime Field $\mathbb{F}_p$)**.** Let $p = 2^{31} - 1 = 2,147,483,647$. Because $p$ is prime, the ring of integers modulo $p$, denoted $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$, forms a finite field of order $p$ under modular addition $+_p$ and modular multiplication $\cdot_p$:

$$a +_p b = (a + b) \bmod p \tag{1}$$

$$a -_p b = (a + p - b) \bmod p \tag{2}$$

$$a \cdot_p b = (a \cdot b) \bmod p \tag{3}$$

for all canonical representatives $a, b \in \{0, 1, \ldots, p - 1\}$.

## 4.1 Overflow-Free 64-Bit Arithmetic Guarantee

In systems programming, modular arithmetic bugs frequently arise from fixed-width integer overflow prior to modular reduction.

**Proposition 4.1** (Absence of `u64` Overflow in $\mathbb{F}_p$)**.** *Let $a, b \in \mathbb{F}_p$ be canonical field elements satisfying $0 \le a, b \le p - 1 = 2^{31} - 2$. When uplifted to unsigned 64-bit integers ($\mathbb{Z} \cap [0, 2^{64} - 1]$), all intermediate addition, subtraction, and multiplication operations execute without overflow or underflow.*

*Proof.* For addition and subtraction, the maximum intermediate value is:

$$a + p = (2^{31} - 2) + (2^{31} - 1) = 2^{32} - 3 < 2^{64} - 1.$$

For subtraction, since $b \le p - 1$ and $a \ge 0$, we have $a + p - b \ge 0 + p - (p - 1) = 1 > 0$, preventing unsigned underflow. For multiplication, the maximum intermediate product occurs at $a = b = p - 1$:

$$(p - 1)^2 = (2^{31} - 2)^2 = 2^{62} - 2^{33} + 4 < 2^{62} < 2^{64} - 1 = \texttt{u64::MAX}.$$

Thus, exact reduction modulo $p$ in `u64` space is strictly free of integer overflow. □

## 4.2 Fermat's Little Theorem and Multiplicative Inverses

Division in $\mathbb{F}_p$ requires computing the unique multiplicative inverse $a^{-1} \in \mathbb{F}_p^*$ for any $a \in \mathbb{F}_p^*$.

**Theorem 4.2** (Fermat's Little Theorem)**.** *For any prime $p$ and any element $a \in \mathbb{F}_p$ with $a \not\equiv 0 \pmod p$:*

$$a^{p-1} \equiv 1 \pmod p.$$

**Corollary 4.3** (Explicit Multiplicative Inverse in $\mathbb{F}_p$)**.** *For every $a \in \mathbb{F}_p^*$, the unique element $a^{-1} \in \mathbb{F}_p^*$ satisfying $a \cdot a^{-1} \equiv 1 \pmod p$ is given by:*

$$a^{-1} \equiv a^{p-2} \pmod p = a^{2,147,483,645} \pmod{2^{31} - 1}.$$

*Proof.* Since $\mathbb{F}_p^*$ is a finite group of order $|\mathbb{F}_p^*| = p - 1$ under modular multiplication, Lagrange's Theorem implies $a^{|\mathbb{F}_p^*|} = a^{p-1} \equiv 1 \pmod p$. Multiplying both sides of the congruence by $a^{-1}$ yields $a^{p-2} \equiv a^{-1} \pmod p$. □

In AEGIS (`src/field.rs`), $a^{p-2} \bmod p$ is computed via binary exponentiation (square-and-multiply). Writing the fixed exponent in binary as $p - 2 = \sum_{i=0}^{30} e_i 2^i$, the computation requires exactly $\lfloor \log_2(p - 2) \rfloor + 1 = 31$ modular squarings and at most 31 modular multiplications ($\mathcal{O}(\log p)$ complexity).

### 4.3 Unbiased Rejection Sampling

To sample $r \xleftarrow{\$} \mathbb{F}_p$ uniformly, reducing a 32-bit random integer $u \in [0, 2^{32} - 1]$ via $u \bmod p$ introduces modulo bias because $p = 2^{31} - 1$ does not divide $2^{32}$. Instead, AEGIS masks 32 CSPRNG bits to 31 bits:

$$u_{31} = u \,\&\, \texttt{0x7FFF_FFFF} \in \{0, 1, \ldots, 2^{31} - 1\}.$$

Since $[0, 2^{31} - 1] = \mathbb{F}_p \cup \{p\}$, rejecting the single boundary outcome $u_{31} = p$ yields an exact uniform distribution over $\mathbb{F}_p$ with single-trial acceptance probability:

$$\Pr[\text{Accept}] = \frac{2^{31} - 1}{2^{31}} = 1 - 2^{-31} \approx 0.99999999953.$$

## 5 Polynomial Construction and Horner's Evaluation

Let $K_{\text{master}} \in \{0,1\}^{256}$ be partitioned into $m = 16$ little-endian 16-bit unsigned integers:

$$s_0, s_1, \ldots, s_{15} \in \{0, 1, \ldots, 2^{16} - 1\}.$$

Since $2^{16} = 65,536 < p = 2,147,483,647$, each chunk embeds injectively into $\mathbb{F}_p$. For each chunk index $r \in \{0, \ldots, 15\}$, we construct a polynomial $f_r(x) \in \mathbb{F}_p[x]$ of degree $k - 1$:

$$f_r(x) = a_{r,0} + a_{r,1}x + a_{r,2}x^2 + \cdots + a_{r,k-1}x^{k-1} \pmod{p} \tag{4}$$

where:

- $a_{r,0} = s_r \in [0, 65535] \subset \mathbb{F}_p$ is the secret chunk embedded at the $y$-intercept $f_r(0) = s_r$,
- $a_{r,1}, \ldots, a_{r,k-2} \xleftarrow{\$} \mathbb{F}_p$ are sampled independently and uniformly from $\mathbb{F}_p$, and
- $a_{r,k-1} \xleftarrow{\$} \mathbb{F}_p^*$ is sampled uniformly from $\mathbb{F}_p \setminus \{0\}$ to guarantee $\deg(f_r) = k - 1$.

To evaluate $f_r(x)$ at participant coordinate $x_i \in \{1, \ldots, n\}$ in $\mathcal{O}(k)$ operations without computing powers $x_i^d$ separately, AEGIS uses **Horner's Method**:

$$f_r(x_i) = a_{r,0} + x_i\Big(a_{r,1} + x_i\big(a_{r,2} + \cdots + x_i \cdot a_{r,k-1}\big)\Big) \pmod{p}. \tag{5}$$

## 6 Lagrange Interpolation at $x = 0$

Given any subset of $k$ shares corresponding to distinct non-zero evaluation abscissas $\{x_0, x_1, \ldots, x_{k-1}\} \subset \mathbb{F}_p^*$, let $y_j = f_r(x_j)$ for $j \in \{0, \ldots, k-1\}$.

**Theorem 6.1** (Lagrange Interpolation Formula over $\mathbb{F}_p$)**.** *There exists a unique polynomial $f_r(x) \in \mathbb{F}_p[x]$ of degree at most $k - 1$ satisfying $f_r(x_j) = y_j$ for all $j \in \{0, \ldots, k-1\}$, given by:*

$$f_r(x) = \sum_{j=0}^{k-1} y_j \cdot \ell_j(x) \pmod{p}, \quad \textit{where} \quad \ell_j(x) = \prod_{\substack{0 \le i < k \\ i \ne j}} \frac{x - x_i}{x_j - x_i} \pmod{p}. \tag{6}$$

Because the secret chunk is located at the constant term $s_r = a_{r,0} = f_r(0)$, AEGIS evaluates the Lagrange basis polynomials directly at $x = 0$:

$$\ell_j(0) = \prod_{\substack{0 \le i < k \\ i \ne j}} \frac{0 - x_i}{x_j - x_i} = \prod_{\substack{0 \le i < k \\ i \ne j}} \frac{x_i}{x_i - x_j} \equiv \left(\prod_{\substack{0 \le i < k \\ i \ne j}} x_i\right) \cdot \left(\prod_{\substack{0 \le i < k \\ i \ne j}} (x_i - x_j)\right)^{p-2} \pmod{p}. \tag{7}$$

Hence, each secret chunk $s_r$ is recovered via:

$$s_r = f_r(0) = \sum_{j=0}^{k-1} y_j \cdot \ell_j(0) \pmod{p}. \tag{8}$$

# 7 Vandermonde Matrix Representation

The uniqueness of polynomial reconstruction in $\mathbb{F}_p[x]$ can be equivalently formulated via linear algebra over the field $\mathbb{F}_p$.

**Theorem 7.1** (Vandermonde Invertibility over $\mathbb{F}_p$)**.** *Evaluating $f_r(x) = \sum_{d=0}^{k-1} a_{r,d}x^d$ at $k$ points $x_0, x_1, \ldots, x_{k-1} \in \mathbb{F}_p$ defines the linear system $V\mathbf{a}_r = \mathbf{y}_r$ over $\mathbb{F}_p$:*

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{k-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{k-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{k-1} & x_{k-1}^2 & \cdots & x_{k-1}^{k-1} \end{bmatrix} \begin{bmatrix} a_{r,0} \\ a_{r,1} \\ \vdots \\ a_{r,k-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{k-1} \end{bmatrix} \pmod{p}. \tag{9}$$

*If $x_0, x_1, \ldots, x_{k-1}$ are pairwise distinct modulo $p$, then the $k \times k$ Vandermonde matrix $V$ is invertible over $\mathbb{F}_p$, and $\mathbf{a}_r = V^{-1}\mathbf{y}_r$ is unique.*

*Proof.* The determinant of the $k \times k$ Vandermonde matrix $V$ is given by the classical product identity:

$$\det(V) \equiv \prod_{0 \le i < j < k} (x_j - x_i) \pmod{p}.$$

Since $x_i, x_j \in \{1, \ldots, 255\} \subset \mathbb{F}_p$ and $x_i \neq x_j$ for all $i \neq j$, every factor satisfies $x_j - x_i \not\equiv 0 \pmod{p}$. Because $\mathbb{F}_p$ is a field (and therefore an integral domain with no zero divisors), a product of non-zero elements in $\mathbb{F}_p$ is strictly non-zero: $\det(V) \not\equiv 0 \pmod{p}$. Consequently, $\det(V)$ possesses a unique multiplicative inverse $(\det V)^{p-2} \in \mathbb{F}_p^*$, proving that $V^{-1}$ exists and the coefficient vector $\mathbf{a}_r = V^{-1}\mathbf{y}_r$ is uniquely determined. □

# 8 Key Derivation Hierarchy (HKDF-SHA256)

To prevent cross-protocol key reuse between threshold algebra over $\mathbb{F}_p$ and symmetric encryption, AEGIS implements RFC 5869 HKDF-SHA256 using the Extract-then-Expand paradigm.

Let $\mathrm{HMAC}(K, m)$ denote HMAC-SHA256 with 64-byte block size $B = 64$:

$$\mathrm{HMAC}(K, m) = \mathrm{SHA256}\Big((K \oplus \mathrm{opad}) \parallel \mathrm{SHA256}\big((K \oplus \mathrm{ipad}) \parallel m\big)\Big). \tag{10}$$

**Stage 1: HKDF-Extract:** Given protocol salt $S_{\mathrm{root}} =$ `"AEGIS-v1-HKDF-SHA256-ROOT-SALT"` and input keying material $K_{\mathrm{master}} \in \{0,1\}^{256}$:

$$\mathrm{PRK} = \mathrm{HMAC}\big(S_{\mathrm{root}}, K_{\mathrm{master}}\big) \in \{0,1\}^{256}. \tag{11}$$

**Stage 2: HKDF-Expand (Orthogonal Domain Separation):** From PRK, AEGIS derives two computationally independent 256-bit outputs using single-block expansion ($L = 32$ bytes):

$$K_{\mathrm{enc}} = \mathrm{HMAC}\big(\mathrm{PRK},\ \texttt{"AEGIS-v1-CHACHA20POLY1305-ENC-KEY"} \parallel \texttt{0x01}\big) \tag{12}$$

$$C_{\mathrm{key}} = \mathrm{HMAC}\big(\mathrm{PRK},\ \texttt{"AEGIS-v1-MASTER-KEY-COMMITMENT"} \parallel \texttt{0x01}\big). \tag{13}$$

Under the standard assumption that HMAC-SHA256 is a secure Pseudorandom Function (PRF), $(K_{\mathrm{enc}}, C_{\mathrm{key}})$ is computationally indistinguishable from two independent uniform random 256-bit strings. Publishing $C_{\mathrm{key}}$ reveals no information about $K_{\mathrm{enc}}$ or $K_{\mathrm{master}}$, while providing a collision-resistant binding commitment to $K_{\mathrm{master}}$.

# 9 Authenticated Encryption (ChaCha20-Poly1305)

AEGIS encrypts payloads using RFC 8439 ChaCha20-Poly1305 with a 256-bit key $K_{\text{enc}}$ and a 96-bit OS CSPRNG nonce $N \xleftarrow{\$} \{0,1\}^{96}$.

## 9.1 ChaCha20 Stream Cipher

ChaCha20 initializes a $4 \times 4$ state matrix of 32-bit little-endian words combining the 128-bit constant $\sigma =$ `"expand 32-byte k"`, the 256-bit key $K_{\text{enc}}$, a 32-bit block counter $c$, and the 96-bit nonce $N$. Block counter $c = 0$ generates the one-time 256-bit Poly1305 key $(r, s) = \text{ChaCha20}(K_{\text{enc}}, 0, N)[0..32]$, while counters $c \geq 1$ produce the keystream XORed with plaintext $P$:

$$C = P \oplus \text{ChaCha20Stream}(K_{\text{enc}}, c \geq 1, N). \tag{14}$$

## 9.2 Canonical AAD Binding and Poly1305 Evaluation

To prevent header stripping or commitment substitution attacks, AEGIS binds the envelope version header, the 32-byte commitment $C_{\text{key}}$, and the user's Associated Data ($\text{AAD}_{\text{user}}$) into an injective canonical AAD transcript:

$$\text{AAD}_{\text{canonical}} = \texttt{"AEGIS-ENVELOPE-v1"} \parallel \texttt{0x00} \parallel C_{\text{key}} \parallel \texttt{0x00} \parallel \text{AAD}_{\text{user}}. \tag{15}$$

Poly1305 pads $\text{AAD}_{\text{canonical}}$, the ciphertext $C$, and their 64-bit little-endian byte lengths into $q$ 16-byte blocks $m_1, \ldots, m_q$ and evaluates a polynomial modulo the prime $2^{130} - 5$:

$$T = \left( \left( \sum_{i=1}^{q} m_i \cdot r^{q-i+1} \right) \bmod (2^{130} - 5) + s \right) \bmod 2^{128}. \tag{16}$$

During decryption, $C_{\text{key}}$ and $T$ are verified using constant-time byte comparisons prior to releasing any decrypted plaintext.

# 10 Secret-Sharing Security Proof & Algebraic Trapdoor

**Theorem 10.1** (Information-Theoretic Security of $(k, n)$ Sharing). *Let $I = \{x_1, \ldots, x_{k-1}\} \subset \mathbb{F}_p^*$ be any subset of $k-1$ distinct share coordinates with observed evaluations $\mathbf{y}_I = (y_1, \ldots, y_{k-1}) \in \mathbb{F}_p^{k-1}$. For any two candidate secrets $s, s' \in [0, 65535] \subset \mathbb{F}_p$:*

$$\Pr[S_r = s \mid \mathbf{Y}_I = \mathbf{y}_I] = \Pr[S_r = s' \mid \mathbf{Y}_I = \mathbf{y}_I] = \Pr[S_r = s].$$

*Proof.* Fix any candidate secret $s^* \in \mathbb{F}_p$ at $x_0 = 0$. The $k$ points $(0, s^*), (x_1, y_1), \ldots, (x_{k-1}, y_{k-1})$ have pairwise distinct abscissas because $x_i \in \{1, \ldots, n\}$ and $x_0 = 0$. By Theorem 6.1 (Vandermonde Invertibility), there exists a unique degree-$(k-1)$ polynomial $f^*(x)$ passing through all $k$ points, which uniquely fixes the $k-1$ random coefficients $(a_1, \ldots, a_{k-1}) \in \mathbb{F}_p^{k-2} \times \mathbb{F}_p^*$. Since the dealer samples $(a_1, \ldots, a_{k-1})$ uniformly and independently of $s_r$, every candidate secret $s^*$ is equally likely given $\mathbf{y}_I$. □

**Proposition 10.2** (Two-Layer Forgery and Corruption Detection Probability). *Suppose an adversary supplies a corrupted share during reconstruction, shifting the interpolated value $f_r(0)$ to a uniformly distributed residue in $\mathbb{F}_p$.*

*(1)* ***Layer 1 (Algebraic Range Trapdoor):*** *A valid chunk must satisfy $0 \leq f_r(0) \leq 2^{16} - 1$. The probability that a single random residue in $\mathbb{F}_p$ passes the range check is $\frac{2^{16}}{2^{31}-1} \approx 2^{-15}$. Across all $m = 16$ chunks, the false-acceptance probability of the range check alone is:*

$$\Pr[\textit{Range Pass}] = \left( \frac{2^{16}}{2^{31} - 1} \right)^{16} \approx 2^{-240} \approx 5.65 \times 10^{-73}.$$

(2) ***Layer 2 (HKDF Cryptographic Commitment):*** *Even if an adversary specifically solves the linear system to force* $f_r(0) \in [0, 2^{16} - 1]$ *for all 16 chunks, the resulting key* $K'_{\text{master}} \neq K_{\text{master}}$ *must match* $C_{\text{key}}$ *under HKDF-SHA256, which fails with probability* $1 - 2^{-256}$.

## 11 Implementation Details in Rust

AEGIS is implemented in pure safe Rust (`#![forbid(unsafe_code)]`) across six modular compilation units:

- `src/error.rs`: Defines `AegisError` with merged authentication error reporting (`AuthenticationFailed`) to eliminate padding/tag distinction oracles.
- `src/keygen.rs`: Implements `MasterKey` with OS CSPRNG entropy (`getrandom`), constant-time equality (`subtle::ConstantTimeEq`), redacted `Debug` formatting, and volatile memory scrubbing (`ZeroizeOnDrop`).
- `src/field.rs`: Implements `FieldElement` over $\mathbb{F}_p = \text{GF}(2^{31}-1)$ using 64-bit intermediate arithmetic, binary exponentiation, Fermat inverses, and Horner's evaluation.
- `src/sss.rs`: Implements $(k, n)$ splitting and Lagrange interpolation at $x = 0$ with duplicate-abscissa and canonical-range checks.
- `src/kdf.rs`: Implements RFC 5869 HKDF-SHA256 domain separation producing `EncryptionKey` and `KeyCommitment`.
- `src/cipher.rs` & `src/engine.rs`: Implements RFC 8439 ChaCha20-Poly1305 and key-committed `AegisEnvelope` workflows (`seal`, `open`, `open_with_shares`).

## 12 Security Analysis

1. **Memory Hygiene:** Secret types omit the `Copy` trait and derive `ZeroizeOnDrop`. Temporary stack arrays (`okm`, `key_bytes`) and heap vectors (`coeffs`, `points`, failed ciphertext buffers) are scrubbed via volatile writes that LLVM dead-store optimization cannot remove.
2. **Timing Side-Channel Resistance:** All secret and tag comparisons execute in constant time via `subtle::ConstantTimeEq`. Fermat inversion $a^{p-2} \bmod p$ iterates over a fixed public exponent $p - 2$, ensuring no secret-dependent loop branches.
3. **Cache Side-Channel Resistance:** Both ChaCha20-Poly1305 (ARX design) and arithmetic over $\mathbb{F}_p$ operate strictly on CPU registers without secret-indexed S-box or log/antilog lookup tables.

## 13 Test Methodology

AEGIS is verified by 46 automated unit, adversarial, Known-Answer Test (KAT), and property-based test suites (`proptest`):

- **Standard KAT Vectors:** Verified against IETF RFC 8439 Section 2.8.2 (ChaCha20-Poly1305) and RFC 5869 (HKDF-SHA256).
- **Exhaustive** $\binom{5}{3}$ **Threshold Reconstruction:** Verifies exact reconstruction across all 10 three-share subsets of a 5-share split.

- **Exhaustive Bit-Flip Fuzzing:** Flips bits across every byte of the 12-byte nonce, ciphertext, 16-byte Poly1305 tag, and 32-byte `KeyCommitment`, asserting 100% rejection.
- **Property-Based Algebra (`proptest`):** Randomly tests field commutativity, associativity, distributivity, Fermat inverses $a \cdot a^{p-2} \equiv 1 \pmod{p}$, and random $(k, n)$ polynomial reconstructions.

## 14 Limitations

(1) **Random 96-Bit Nonce Birthday Bound:** Sampling 96-bit nonces uniformly via CSPRNG bounds safe invocations under a single `MasterKey` to approximately $2^{32}$ messages (collision probability $\approx 2^{-33}$). High-volume deployments exceeding $2^{32}$ messages should rotate keys or use 192-bit nonces (XChaCha20-Poly1305).

(2) **Microarchitectural Division Latency:** While exponentiation in $\mathbb{F}_p$ uses a fixed public exponent, modular reduction uses hardware integer division (`%`), which can exhibit minor operand-dependent latency on certain CPU microarchitectures unless replaced by bitwise Mersenne folding $(x \,\&\, p) + (x \gg 31)$.

(3) **Host OS and Register Spills:** Software-level `zeroize` scrubs stack and heap allocations but cannot wipe internal CPU register spills or unencrypted OS swap pages if `mlock(2)` is not enabled.

## 15 Conclusion

AEGIS demonstrates that combining prime finite-field threshold algebra over GF($2^{31} - 1$), HKDF-SHA256 key commitments, and ChaCha20-Poly1305 AEAD in memory-safe Rust yields a high-assurance cryptographic architecture. By uniting an algebraic range trapdoor $(1 - 2^{-240})$ with a constant-time HKDF commitment $(1 - 2^{-256})$ and canonical AAD binding, AEGIS eliminates both single-key storage fragility and unauthenticated secret reconstruction.

## References


[1] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.

[2] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," IETF RFC 5869, May 2010.

[3] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," IETF RFC 8439, June 2018.